# Health Insurance Cross-Sell Prediction

## Machine Learning Project Work

Giacomo Piergentili

2025/2026

# Problem Statement

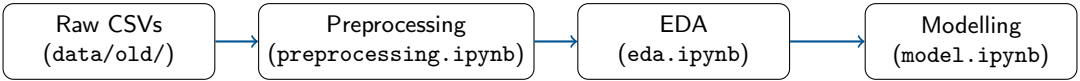
## Business context

An insurance company wants to identify its **health insurance** customers who are also likely to purchase **vehicle insurance**.

- **Goal:** predict whether a customer will respond positively to a cross-sell offer ($\texttt{Response} \in \{0, 1\}$)
- **Metric:** ROC-AUC
- **Dataset:** $\sim$11.5 M labelled customer records
- **Source:** Kaggle Playground Series, Season 4, Episode 7

Original dataset: *Health Insurance Cross Sell Prediction* by Anmol Kumar (Kaggle).

# Project Pipeline




- All intermediate data stored as **Parquet** files (columnar, compressed, dtype-preserving)
- Reproducible: each notebook reads the output of the previous one

# Dataset Overview

| Feature | Type | Description |
|---|---|---|
| `Age` | int | Customer age (20–85) |
| `Gender` | cat | Male / Female |
| `Driving_License` | bin | 1 = holds a licence |
| `Region_Code` | cat | 53 distinct regions (0–52) |
| `Previously_Insured` | bin | 1 = already has vehicle insurance |
| `Vehicle_Age` | ord | <1 yr / 1–2 yr / >2 yr |
| `Vehicle_Damage` | bin | 1 = vehicle was previously damaged |
| `Annual_Premium` | int | Amount paid for health insurance |
| `Policy_Sales_Channel` | cat | 152 distinct sales channels |
| `Vintage` | int | Days since the customer joined |
| `Response` | bin | **Target**: 1 = interested |

# Data Inspection & Encoding

## Quick checks

- $\sim$11.5 M rows $\times$ 12 columns
- No missing values, no duplicates
- 3 columns stored as strings that need encoding

## Problems found

- `Gender`, `Vehicle_Age`, `Vehicle_Damage` stored as strings
- Most numeric columns use `int64`/`float64` (way bigger than needed)

## Encoding applied

| Column | Mapping |
|---|---|
| `Gender` | Male$\rightarrow$0, Female$\rightarrow$1 |
| `Vehicle_Age` | $<$1yr$\rightarrow$0, 1–2yr$\rightarrow$1, $>$2yr$\rightarrow$2 |
| `Vehicle_Damage` | No$\rightarrow$0, Yes$\rightarrow$1 |

`Region_Code` and `Policy_Sales_Channel` were already integers.

# Type Optimisation & Export

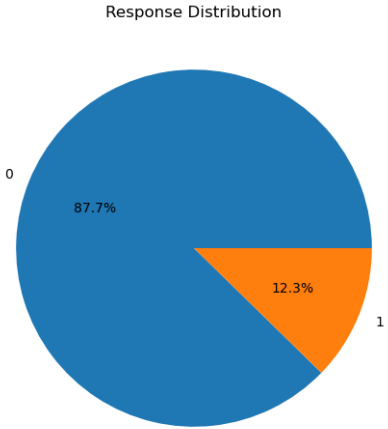
Downcast all columns to the smallest integer type that fits:

| Column(s) | Before | After |
|---|---|---|
| Age, Gender, etc. | `int64` / `object` | `int8` |
| Annual_Premium | `float64` | `int32` |
| Policy_Sales_Channel | `float64` | `int16` |
| Vintage | `int64` | `int16` |

Saved the processed DataFrames as **Parquet** files in `data/processed/` for the next notebooks.

# Class Imbalance

- 87.7% not interested, only 12.3% interested
- Pretty imbalanced, so we need to handle this during training
- Used `class_weight='balanced'` for LR and `is_unbalance=True` for LightGBM



# Feature Distributions

### Gender

- Roughly balanced
- Probably not that useful on its own

### Age

- Skewed right
- Most customers are 20–50
- Tail goes up to 85

### Vehicle Age

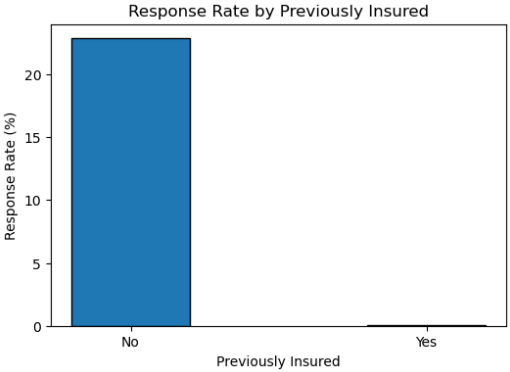
- Most cars are 1–2 years old
- New cars ($<$1yr) almost never have damage
- Older cars $\rightarrow$ more damage

### Annual Premium

- Very skewed, lots of outliers
- Big spike at low premiums
- Clipped top 1% for plots

# Previously Insured: Strongest Predictor

- If a customer already has vehicle insurance, they basically never respond
- Not insured: ∼22% response rate
- Already insured: ∼0% response rate
- This one feature pretty much determines the outcome



# Vehicle Damage & Vehicle Age

**Vehicle Damage**

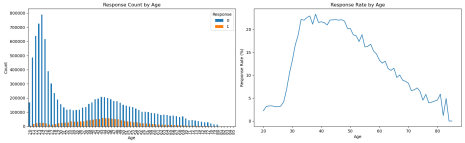
- Prior damage: response $\approx$ 24%
- No damage: response $\approx$ 1%
- Makes sense: if your car was damaged before, you probably want insurance

**Vehicle Age**

- $<$1 year: low response
- 1–2 years: moderate
- $>$2 years: highest response
- Older car = more reason to get coverage

# Response by Customer Age

- Response rate is highest for ages 30–50
- Younger customers (20–29) are less interested even though there are lots of them
- 70+ also lower response



# Sales Channel & Region

**Policy Sales Channel**

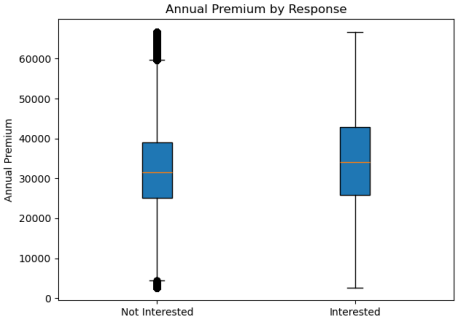
- 152 distinct channels
- Response rate varies a lot across channels
- A few big channels dominate the volume

**Region Code**

- 53 distinct regions
- Most cluster around $\sim$10% response rate
- Some regions are quite different from the average

# Annual Premium by Response

- Distributions overlap a lot between interested and not interested
- Medians are similar, so premium alone isn't a great predictor
- There's a spike at low premiums that tree models might pick up on
- Removed top 1% outliers for the plot



# Correlation Analysis

Pearson correlation heatmap. Two pairs stand out:

| **Feature pair** | $r$ | **What it means** |
|---|---|---|
| `Previously_Insured` & `Vehicle_Damage` | $-0.84$ | Almost the same info (inverted) |
| `Age` & `Vehicle_Age` | $+0.78$ | Older customers tend to have older cars |

Everything else is weakly correlated. These two pairs are what motivated the feature engineering experiment later.

# Correlation: Closer Look

**Previously Insured vs. Vehicle Damage**

$r \approx -0.84$

- Already insured $\Rightarrow$ almost never flagged as damaged
- They carry basically the same information (inverted)
- Could try dropping one of them

**Age vs. Vehicle Age**

$r \approx +0.78$

- 20–29 age group: 97% own cars $<$1 year
- Older groups: more spread out across vehicle ages
- This sharp break suggests trying an `Age` $\times$ `Vehicle_Age` interaction

# EDA Summary

Main findings:

- Only 12.3% positive, so we need to handle class imbalance
- `Previously_Insured` is by far the strongest predictor
- `Vehicle_Damage` is useful but almost redundant with it ($r = -0.84$)
- Older vehicles and ages 30–50 show higher response rates
- Sales channels vary a lot in conversion
- Two strong correlations suggest trying some feature engineering

# Modelling Setup

**Two models**

- **Logistic Regression** (linear baseline) `class_weight='balanced'`, standardised features
- **LightGBM** (gradient-boosted trees) `is_unbalance=True`, 500 trees, lr $= 0.05$

**Two feature sets**

- **Original**: all 10 columns as-is
- **Engineered**: drop `Vehicle_Damage`, frequency-encode `Region_Code` & `Policy_Sales_Channel`, add `Age` $\times$ `Vehicle_Age`

**How I evaluated**

- 80/20 stratified split (same split for both feature sets)
- Main metric: ROC-AUC
- Also looked at Precision, Recall, F1 (threshold $= 0.5$)

**Why the feature engineering**

- Motivated by the EDA correlations ($r = -0.84$, $r = +0.78$)
- Frequency maps computed on train split only (no leakage)

# Results

| Model | ROC-AUC | Precision | Recall | F1 |
|---|---|---|---|---|
| LR (original) | 0.8365 | 0.2526 | 0.9801 | 0.4017 |
| LR (engineered) | 0.8222 | 0.2336 | 0.9739 | 0.3769 |
| LightGBM (original) | **0.8784** | 0.3000 | 0.9296 | 0.4536 |
| LightGBM (engineered) | 0.8693 | 0.2872 | 0.9255 | 0.4384 |

- LightGBM beats LR across the board
- Engineered features actually made things worse for both models
- Recall is high everywhere thanks to class weighting
- Precision is low because of the 88/12 imbalance (at threshold 0.5)

# Why Feature Engineering Didn't Help

1. **Trees already handle this stuff**
   LightGBM can learn non-linear splits, deal with high-cardinality features, and find interactions on its own, so the manual transforms were redundant.

2. **Dropping `Vehicle_Damage` lost some signal**
   Even with $r \approx -0.84$ vs `Previously_Insured`, the leftover information was still useful for some tree splits.

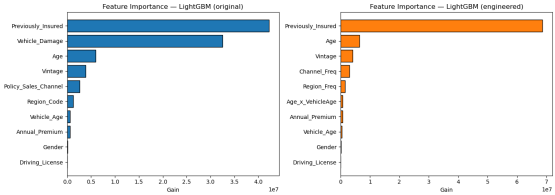
3. **Frequency encoding threw away information**
   Going from 53 distinct region codes to a single number lost the fine-grained differences between regions.

Basically, feature engineering that helps linear models can actually hurt tree models that already handle these patterns internally.

# Feature Importance

## Most important features (by gain)

1. `Previously_Insured`, way ahead of everything else
2. `Vehicle_Age`
3. `Annual_Premium`
4. `Policy_Sales_Channel`
5. `Region_Code` / `Age`




Lines up well with what the EDA showed: `Previously_Insured` dominates.

# Conclusion & Limitations

**What worked**

- LightGBM on original features was the best model (AUC = 0.878)
- Class weighting handled the imbalance well, with high recall across the board
- EDA findings were confirmed by the model (`Previously_Insured` dominates)

**What could be improved**

- Only used a single train/val split; k-fold would be more robust
- Didn't tune the classification threshold (just used 0.5)
- Could try other models like CatBoost or XGBoost
- Target encoding might work better than frequency encoding for high-cardinality features